

Gestion de Projet

Rapport Voiture

Année Académique 2024-2025

Groupe N°7

Genart Alex

Gillard Robin

Noel Charles

Pischedda Noa

Rishing Silva Bryam

Thiry Liam

Table des matières

1. Introduction.....	4
2. Présentation générale du projet	4
3. Présentation détaillée des composants	4
3.1. Les capteurs ultrason	4
3.2. Les capteurs de ligne	5
3.3. Le raspberry PI.....	6
3.4. Capteur de courant (INA219)	7
3.5. Capteur couleur (TCS 3472)	8
3.6. Motor driver (L298N)	9
3.7. Servomoteur.....	10
3.8. Contrôleur PWM (PCA 9685)	11
3.9. Alternatives au Raspberry Pi	12
3.10. Schéma de câblage de tous les composants	12
4. Présentation des méthodes Scrum et de notre gestion de projet	13
4.1. Introduction.....	13
4.2. Rôles	13
4.3. Implémentation.....	13
5. Algorithmes de programmation	15
5.1. Diagramme de classes	15
5.2. Explication des Classes	15
5.3. Algorithme Initial.....	17
5.4. Explication du fonctionnement.....	18
6. Problèmes rencontrés	21
7. Améliorations et conclusions	22
8. Bibliographie	23
9. Annexes	23
9.1. Programmes complet.....	23
9.2. Gestion du projet	24

1. Introduction

Dans le cadre du cours de gestion de projet nous avons eu un projet dont le but est de réaliser une voiture autonome capable de suivre un circuit imposé au préalable. Le véhicule que nous utiliserons durant ce projet, a déjà été monté au préalable et est basé sur un kit SunFounder le tout contrôlé par un Raspberry PI modèle 3B. L'objectif à atteindre aux termes de ce projet est de répondre aux attentes du client qui nous demande de réaliser un tour de circuit le plus rapidement possible afin de se qualifier pour la course qui opposera les différents groupes.

Notre approche du projet était orientée en test first. En effet, nous avons réalisé une grande quantité de tests unitaires afin de s'assurer que les capteurs fonctionnaient et afin de ne pas abimer le matériel. Ces tests nous ont aussi permis de s'assurer que notre code répondait bien aux exigences du client.

Notre code était entièrement réalisé en Python et orienté objet. L'approche du projet en programmation objet nous a permis de mieux structurer notre code à l'aide de classes et de méthodes

2. Présentation générale du projet

Lorsque nous avons commencé la semaine de projet, nous avons reçu un cahier des charges afin de nous définir des objectifs précis tel qu'éviter des obstacles, détecter des feux de départ, s'arrêter après avoir franchi la ligne d'arrivée mais surtout réaliser un tour complet de la piste imposée par les enseignants.

Ce projet nous a permis de développer notre utilisation pratique de la méthode SCRUM ainsi que de développer notre approche test first lors des projets. Nous avons également renforcé notre compréhension de l'utilisation et la programmation de systèmes ainsi que l'application de la théorie liée aux composants de la voiture.

Ce projet nous permet également d'approfondir nos connaissances sur différents domaines tels que l'électronique avec l'inspection du câblage, la programmation en rédigeant les tests unitaires, la modélisation UML qui nous a permis de mieux orienter la rédaction de nos classes et méthodes et pour finir la rédaction du code qui nous permettra de faire évoluer la voiture dans les différents virages qui composent le circuit.

3. Présentation détaillée des composants

3.1. *Les capteurs ultrason*

Utilité

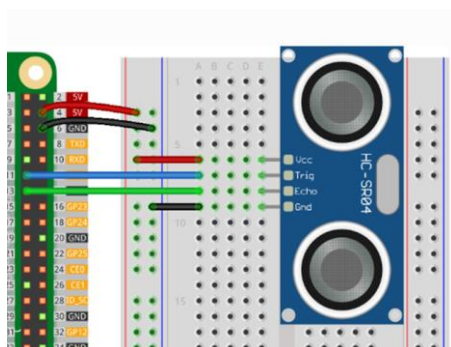
Le capteur à ultrason permet de détecter les obstacles et de mesurer les distances entre ceux-ci et le capteur. La voiture étant équipée de 3 capteurs (devant, gauche et droite), la voiture peut gérer la distance avec des obstacles qu'elle croiserait sur sa route et les parois du circuit sur lequel elle progresse.

4

Câblage

Le capteur est composé de 4 broches :

- VCC : alimentation 5V
- Trig : signal de déclenchement
- Echo : signal de retour
- GND : masse.



Fonctionnement

Le capteur ultrason envoie une impulsion de 10 microsecondes à la broche Trig, ce qui déclenche l'émission d'un ultrason à 40kHz émis par le capteur. L'onde se propage dans l'air jusqu'au moment où elle rencontre un obstacle. L'onde rebondit sur l'obstacle et revient vers le capteur, elle est alors captée par la broche ECHO qui est en état haut durant le trajet de l'onde. Le capteur mesure la durée de l'état haut de la broche ECHO. La distance est donc calculée avec la formule suivante :

$$\text{Distance (cm)} = \frac{\text{durée } (\mu\text{s}) \cdot 0.0343}{2}$$

3.2. Les capteurs de ligne

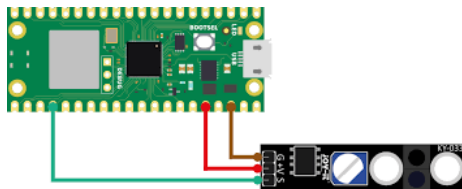
Utilité

Le capteur sert à détecter le franchissement de la ligne de départ/arrivé du circuit afin de compter le nombre de tours effectués, calculer le temps que la voiture a mis pour effectuer un tour de circuit ou encore de s'arrêter après avoir franchi la ligne de départ/arrivé.

Câblage

Le capteur est équipé de trois broches :

- VCC : alimentation de 3,3V à 5V
- GND : masse
- DO (Digital Output) : renvoi un état haut si le signal infrarouge est capté et un état bas dans le cas contraire.



Fonctionnement

Le capteur de ligne équipé d'une LED infrarouge va éclairer le sol. Quand il y a une surface blanche qui est éclairée, la lumière est réfléchiée et captée par le phototransistor. La sortie DO est alors haute (ou basse selon la configuration du capteur). Lorsqu'une surface noire est éclairée, il y a peu voire plus de réflexion, l'état de DO s'inverse alors.

3.3. Le raspberry PI

Utilité

Le Raspberry Pi est essentiel au projet de la voiture autonome. En effet, il est le cerveau de la voiture étant donné qu'il va interfacer tous les composants et exécuter le code Python pour la navigation de la voiture. Il communique avec les moteurs et les capteurs afin de que la voiture se déplace dans l'environnement en prenant compte des éventuels obstacles qui pourraient se présenter sur le circuit.

Câblage

Le Raspberry Pi est équipé en 5V et est composé d'un processeur 4 cœurs ARM Cortex-A53 ARM v8 64bits et cadencé à 1.4GHz

Le Raspberry Pi possède les connectivités suivantes :

- 4 interfaces USB 2.0
- 1 interface gigabit Ethernet via USB 2.0
- Un réseau local sans fil IEEE 802.11b/g/n/ac 2.4GHz et 5GHz
- Bluetooth 4.2 et Bluetooth Low Energy

Ainsi que les ports suivants :

- Un port HDMI full size
- Un Port CSI pour caméra
- Un port DSI pour écran tactile
- Sortie stéréo 4 pôles et port vidéo composite
- Un emplacement micro SD
- Un port jack
- Un connecteur GPIO étendu de 40 broches

3.4. *Capteur de courant (INA219)*

Utilité

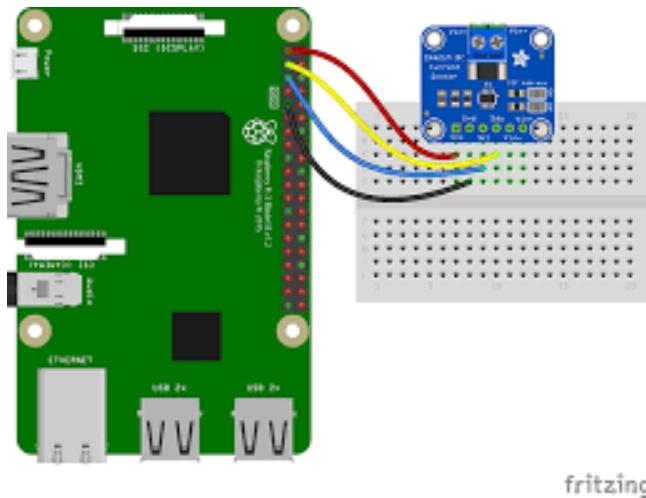
Le capteur de courant nous permet de mesurer précisément le courant qui passe à travers le servomoteur. Grâce à ce capteur, nous pouvons si une mesure anormale de courant alimente le servomoteur et pourrais l'endommager.

Câblage

Le capteur de courant utilise le bus de communication i²C, un bus asynchrone et bidirectionnel qui utilise un premier fil dédié au signal d'horloge, qui synchronise les transmissions et un autre qui échange les données entre les périphériques.

Les broches du capteur de tension sont les suivantes :

- VCC: alimentation de 5V
- GND: masse
- SCL (Serial Clock): signal d'horloge qui synchronise les transmissions
- SDA (Serial Data): il sert à transmettre les données
- Vin +: pour l'arrivée du courant
- Vin -: pour la sortie du courant vers le composant



Fonctionnement

Le capteur mesure le courant en utilisant une résistance shunt placée en série avec la charge. Le courant qui passe génère une chute de tension aux bornes. Le capteur peut mesurer précisément la différence de tension.

3.5. Capteur couleur (TCS 3472)

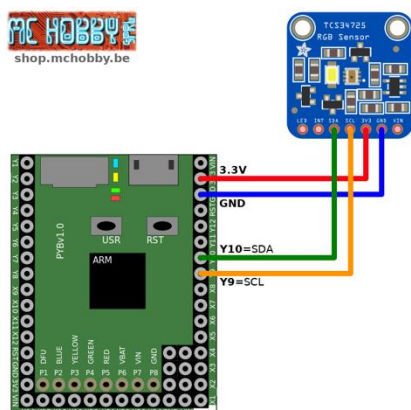
Utilité

Le capteur de couleur permet de capter le signal lumineux vert de départ et de transmettre par la suite l'information au Raspberry qui permettra à la voiture de démarrer.

Câblage

Le capteur de couleur communique sur le bus sur I²C et est composé de broches suivantes :

- VCC: alimentation en 5V
- GND: masse
- 3,3V: permet l'alimentation en 3,3V
- SCL(Serial Clock) : signal d'horloge qui permet de synchroniser la transmission de données.
- SDA (Serial Data) : permet de transmettre les données
- INT: signal d'interruption
- LED: contrôle de la LED



Fonctionnement

Le capteur de couleur est équipé de quatre photodiodes qui captent le rouge, le bleu, le vert et la lumière claire grâce aux longueurs d'ondes. Lorsque les UV sont filtrés, chaque photodiode envoie l'information vers le Raspberry Pi, qui traduit l'information analogique en donnée numérique grâce au convertisseur analogique-numérique (ADC)

3.6. Motor driver (L298N)

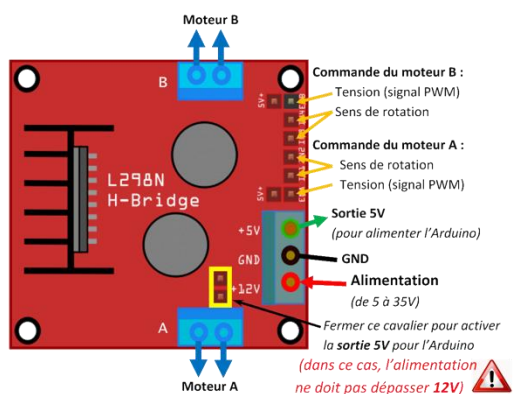
Utilité

Le Motor driver va permettre aux deux moteurs de la voiture d'éviter une surtension. Il gère également de manière précise la vitesse et la direction des moteurs.

Câblage

Le câblage du Motor driver comporte plusieurs connexions importantes, l'alimentation principale et les deux sorties des moteurs. Il est aussi composé d'un ensemble de broches de contrôle des moteurs :

- ENABLE: une broche par moteur qui permet de gérer la vitesse de chaque moteur via un signal PWM
- IN: deux broches par moteur qui permettent de choisir le sens du moteur.



Fonctionnement

Le L298N est un montage de transistors qui connectent les moteurs à leur alimentation. Pour contrôler le moteur, le L298N requiert un regroupement de deux informations répartis sur trois broches. Chaque moteur a besoin d'un signal PWM sur la broche ENABLE qui représente la vitesse de rotation d'un moteur (le signal va de 0 à 4095). Le sens de rotation du moteur est indiqué grâce aux états haut ou bas des broches IN1 et IN2.

3.7. Servomoteur

Utilité

Le servomoteur est l'un des éléments les plus importants qui jouent un rôle dans le déplacement du véhicule. En effet, c'est le servomoteur qui va gérer la direction des roues avant. En fonction des signaux PWM qu'il reçoit, le servomoteur peut faire tourner les roues à un angle précis.

Câblage

Le servomoteur fonctionne grâce à trois fils :

- Orange : signal de contrôle PWM
- Rouge : alimentation en 5V
- Brun : masse



Fonctionnement

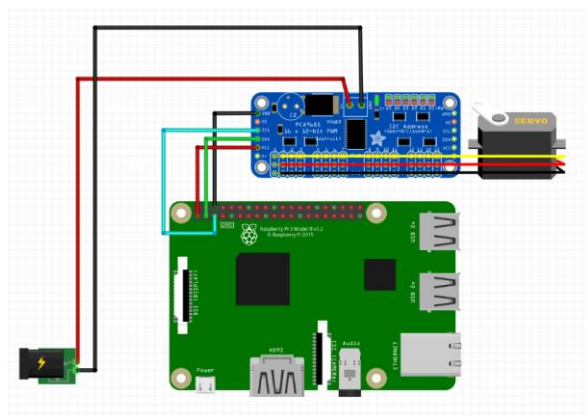
Le servomoteur est un dispositif électromagnétique qui transforme un signal PWM en un mouvement à amplitude précise. Il est équipé d'un moteur sur lequel est monté un dispositif qui diminue la vitesse du moteur mais augmente son couple. La précision du servomoteur repose sur une boucle de rétroaction qui permet de contrôler l'angle en l'ajustant afin d'atteindre la position souhaitée par rapport à sa position initiale grâce à un potentiomètre situé en dessous de l'arbre de sortie. Selon l'impulsion du signal PWM, le servomoteur tournera d'autant de degrés par rapport à sa position initiale.

3.8. Contrôleur PWM (PCA 9685)

Utilité

Il permet de contrôler simultanément les composants qui utilisent un signal PWM, c'est-à-dire le servomoteur et la vitesse des moteurs.

Câblage



Le contrôleur PWM nécessite sa propre alimentation via la borne supérieure afin d'alimenter les composants qui y sont directement connectés. Etant contrôlable via les bus I²C, il nécessite que les fils SDA et SCL soient connectés sur le bus

Fonctionnement

Pour contrôler le PCA 9685, il est nécessaire d'importer une bibliothèque dans le code. Pour contrôler les différentes sorties PWM, il faut envoyer les instructions à son adresse I²C la sortie PWM souhaitée.

3.9. Alternatives au Raspberry Pi

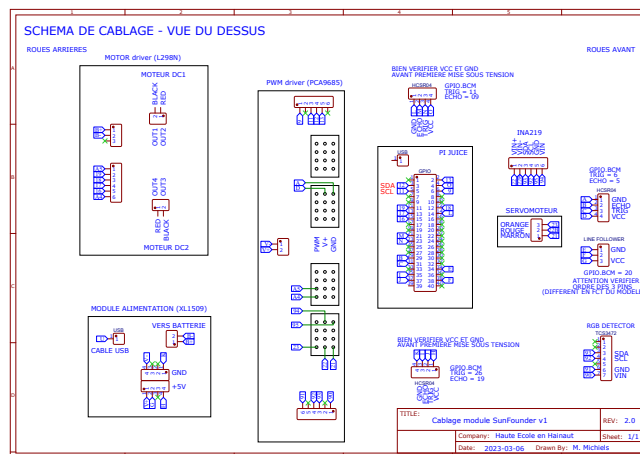
Il est nécessaire d'explorer les différentes alternatives au Raspberry Pi. Nous pouvons justifier notre choix à travers ce tableau comparatif :

En conclusion de ce tableau comparatif entre différents nano-ordinateurs, nous pouvons observer que les deux plus intéressants sur le rapport qualité/prix sont le Raspberry Pi 3 et le Orange Pi 3. En effet, les deux nano-ordinateurs sont assez similaires. Ils se différencient par leur mémoire vive, légèrement sur leur prix mais la différence la plus notable est la large communauté du Raspberry Pi qui n'est pas aussi grande pour le

Orange Pi. C'est ce dernier critère qui a orienté notre choix sur le Raspberry Pi 3. Ce choix a été fait en prévision du projet car nous étions sûr de se retrouver face à de la difficulté et avons donc voulu avoir la communauté du Raspberry Pi pour nous aider à trouver des solutions pour satisfaire au mieux les demandes du client.

	PRIX	ARCHITECTURE PROCESSEUR	SPEC. PROCESSEUR	RAM	CONNECTIVITE
RASPBERRY PI 3B	++ 45€	ARM Cortex-A53	1.4ghz 4 coeurs	1 Go	Wifi, BT, Ethernet, USB2.0
ESP32	++ 20€	Xtensa LX6	Up to 240Mhz 2 coeurs	520 Ko	Wifi, BT
ORANGE PI 3B	++ 50€	ARM Cortex-A53	1.8ghz 4 coeurs	2 Go	Wifi, BT, Ethernet, USB2.0
JETSON NANO	++ 200€	ARM Cortex-A57	1.4ghz 4 coeurs	4 Go	Wifi, BT, Ethernet, USB3.0
	STOCKAGE	CONSO. ENERGIE	TAILLE	OS	COMMUNAUTE
RASPBERRY PI 3	microSD	2.5 -> 4 W	85.60 mm x 56.5 mm x 17 mm.	Raspberry PI OS	+++
ESP32	mémoire flash	30mA -> 70mA	27.5mm x 58mm x 5mm	FreeRTOS	++
ORANGE PI 3	microSD possibilité SATA/eMMC	2.5 -> 4 W	56 x 85 mm	Orange PI OS	+
JETSON NANO	microSD SSD via USB 3.0	5 W	100 mm x 80 mm.	Linux	++

3.10. Schéma de câblage de tous les composants



4. Présentation des méthodes Scrum et de notre gestion de projet

4.1. Introduction

Scrum est une méthodologie Agile qui permet de gérer des projets complexes de manière flexible, en divisant les tâches les plus complexes et en nous les répartissant.

On a décidé d'utiliser SCRUM pour faciliter la gestion de notre projet, la conception d'une voiture autonome capable d'éviter des obstacles et utiliser un ensemble de capteurs pour interagir avec son environnement.

4.2. Rôles

Product Owner :

Le product Owner est le client c'est lui qui définit les besoins.

Scrum Master : Bryam Rishing.

Le SCRUM Master est la personne qui facilite la mise en place de la méthodologie SCRUM en enlevant les obstacles et est le pont entre les développeurs et le client.

L'équipe de développement :

Genart Alex, Gillard Robin, Noel Charles, Pischedda Noa, Rishing Silva Bryam, Thiry Liam
Ce sont les personnes qui ont travaillé activement sur le produit final.

4.3. Implémentation

User Stories

Les user stories sont un outil central dans les méthodes agiles, notamment Scrum.

Elles permettent de décrire simplement les besoins d'un utilisateur ou d'un client sous forme de petites fonctionnalités à développer.

Product backlog

Le Product Backlog est la phase de rencontre entre nous et le client c'est ici que l'on a analyser son besoin et que l'on tire les User Stories, pour ce faire on a mis en place un Trello car comme dans SCRUM le cahier de charges n'est pas définitif et peut changer en fonction des besoins du client.

Pendant cette phase on a mis une priorité aux tâches et sélectionner celles qu'on allait réaliser dans le premier Sprint.

User Stories Sélectionnés :

N°	Qui	Quoi
1	Superviseur	Pouvoir se connecter à distance au module
2	Superviseur	Avoir une doc sur nos choix techniques

3	Mécanicien	Tester si les capteurs et moteurs marchent bien
5	Pilote	Voir la vitesse et l'état du module en direct
6	Mécanicien	Accéder aux logs quand y'a un problème
7	Pilote	Faire des demi-tours dans des petits espaces
8	Directeur	Redémarrer la voiture à distance si elle plante
9	Pilote	Détecter et éviter les obstacles
10	Pilote	S'arrêter pile sur la ligne d'arrivée (à 5cm près)
11	Pilote	Optimiser les trajectoires pour aller plus vite
12	Directeur	Choisir combien de tours la voiture doit faire
13	Directeur	Démarrer auto quand y'a le signal
14	Équipe	Faire un tour complet
15	Équipe	Participer à une vraie course.

Sprints

Nos Sprints on durer 2 jours durant lesquelles on a utilisé GitHub afin d'avoir des incréments finis et fonctionnels dans une branche principale.

Sprint planning

Le Sprint Planning a pour but de définir le prochain sprint. Le scrum master et le reste de l'équipe se réunissent afin de se mettre d'accord sur les éléments du Product Backlog qui seront inclut dans le sprint suivant pour atteindre un objectif et sur la manière d'y parvenir.

Dans le premier Sprint on s'est focaliser sur les User Stories de 1 à 4.

Dans le deuxième on a fait les User Stories du 2 au 13.

De plus on a utilisé le planning poker pour estimer la complexité de chaque tâche.

Voir point 9.2 Gestion de projet, annexe 1 Sprint planning.

Daily SCRUM

Le Daily SCRUM est la partie qui sert à mettre l'équipe au courant des avancées blocages et problèmes rencontrés dans l'optique de nous entre aider et avancer sur le projet.

Le Daily Scrum ne dure pas plus que 10 minutes pour garantir l'efficacité de l'équipe.

Sprint Review

Pendant le Sprint Review on valide les incréments avec le Product Owner. C'est le moment ou on reçoit un feedback du client.

Sprint Rétrospective

Le Sprint Rétrospective nous a permis d'analyser notre manière de travailler et d'identifier :

Les pratiques efficaces à conserver

Les difficultés rencontrées

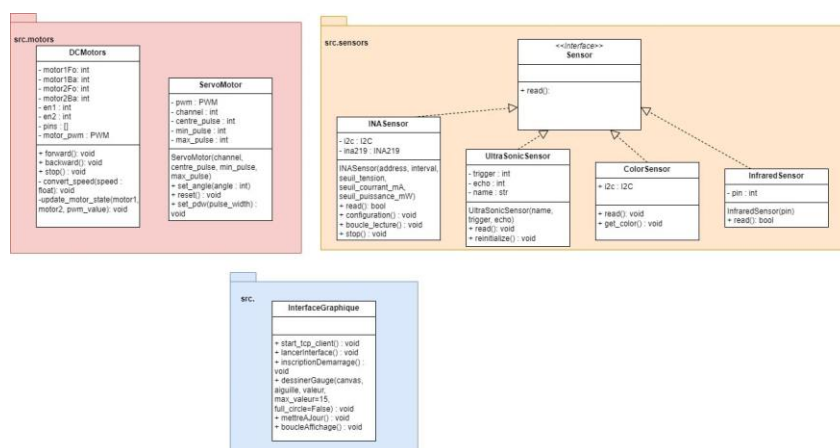
Les améliorations que l'on a apporté au processus de développement

Voir point 9.2 Gestion de projet, annexe 2 Sprint Retrospective.

5. Algorithmes de programmation

5.1. Diagramme de classes

Dans le cadre du projet, nous avons conçu un diagramme de classes afin d'obtenir une vision structurée et plus précise des différentes classes, de leurs liens et de leurs interactions. Ce schéma nous a permis de mieux organiser le développement, facilitant ainsi la conception, la maintenance et les potentiels évolutions.



5.2. Explication des Classes

Classe `Sensor` :

L'interface `Sensor` définit une méthode commune `read()`, que chaque capteur concret doit implémenter. Cette méthode permet de récupérer les données spécifiques à chaque type de capteur.

Classes concrètes de capteurs :

Classe `INASensor` :

La classe `INASensor` représente un capteur de courant et de tension basé sur le module INA219.

Attributs principaux :

- i2c : interface de communication I2C utilisée pour échanger avec le capteur.
- ina219 : instance du module de mesure INA219.

- **Méthodes principales :**

- configure() : Permet la configuration du capteur.
- boucle_lecture() : Permet de stocker différentes données comme la tension, le courant et la puissance pendant la lecture du capteur.
- stop() : Permet d'arrêter le capteur
- read() : Affiche différentes données (courant, tension, puissance).

Classe 'UltraSonicSensor' :

- **Attributs principaux :**

- trigger: broche utilisée pour émettre l'impulsion ultrasonique.
- echo: broche utilisée pour recevoir le signal réfléchi.
- name: identifiant du capteur (ex. "avant", "gauche", "droite") afin de le situer sur le véhicule.

- **Méthodes principales :**

- reinitialize(): réinitialise le capteur.
- read(): mesure et stocke la distance détectée.

Classe 'ColorSensor' :

- **Attributs principaux :**

- i2c : interface de communication I2C utilisée pour échanger avec le capteur.

- **Méthodes principales :**

- get_color(): lit les valeurs RGB captées par le capteur et vérifie si la couleur verte est détectée. La méthode retourne True en cas de détection du vert, sinon False.
- read(): Retourne les valeurs RGB (0-255) mesurées.

Classe 'InfraredSensor' :

- **Attributs principaux :**

- pin: broche du Raspberry Pi utilisée pour connecter le capteur infrarouge.

- **Méthodes principales :**

- read(): lit et retourne l'état du capteur infrarouge (ligne d'arrivée détectée ou non).

Classe 'DCMotor' :

La classe DCMotor est chargée de gérer le moteur de propulsion du véhicule. Elle offre des méthodes de contrôle direct telles que forward pour faire avancer la voiture, backward pour la faire reculer, et stop pour arrêter le moteur.

En interne, la classe utilise des méthodes privées pour gérer la conversion et l'application des vitesses moteur. La méthode convert_speed convertit une valeur de vitesse exprimée entre -100 et 100 en une valeur PWM comprise entre 0 et 4095. La méthode update_motor_state met

à jour l'état du moteur en contrôlant la direction (via des broches GPIO) et la vitesse (via un signal PWM).

Classe 'ServoMotor' :

La classe ServoMotor est responsable du contrôle du servomoteur, qui permet d'orienter les roues de la voiture. Elle offre plusieurs méthodes pour ajuster la direction :

- `set_angle` permet de définir un angle précis pour orienter les roues selon les besoins de navigation.
- `reset` recentre les roues, les ramenant à la position "tout droit".
- `set_pdw` définit manuellement la largeur d'impulsion (PWM) envoyée au servomoteur, afin de régler précisément sa position.

Classe 'InterfaceGraphique' :

La classe InterfaceGraphique est chargée de gérer l'affichage de l'interface utilisateur ainsi que la communication réseau avec la voiture. Elle permet notamment de lancer la connexion TCP avec le Raspberry Pi, d'afficher l'interface, de la mettre à jour en temps réel avec les données reçues (comme les distances mesurées ou l'état des capteurs), et de la fermer proprement en fin d'exécution. Cette interface offre une visualisation pratique du comportement du véhicule pendant son fonctionnement.

5.3. *Algorithme Initial*

Lors de la phase de conception, nous avons imaginé un algorithme permettant d'adapter dynamiquement la direction de la voiture autonome en fonction des obstacles détectés à l'aide de capteurs à ultrasons. Trois capteurs étaient utilisés : un positionné à l'avant, un à gauche et un à droite.

Lorsque le capteur frontal détectait un obstacle à une distance inférieure à un seuil défini (par exemple 20 cm), l'algorithme se déclenchait afin de déterminer dans quelle direction la voiture devait tourner pour éviter la collision. Pour ce faire, la différence entre la distance mesurée à droite et celle mesurée à gauche était calculée :

```
écart = distance_droite - distance_gauche
```

Ce delta permettait d'estimer de quel côté l'espace libre était le plus grand. Un écart positif indiquait une ouverture plus large à droite, suggérant donc de tourner dans cette direction. L'angle à appliquer au servomoteur (chargé de l'orientation des roues) était alors déterminé selon la formule suivante :

```
angle = -10 + (écart * facteur)
```

Ici, -10 correspond à la position "tout droit", en fonction du calibrage du servomoteur, dont l'amplitude de fonctionnement allait de -35° (gauche maximum) à +15° (droite maximum). Le

facteur jouait le rôle de coefficient d'ajustement, déterminé à partir des tests réalisés sur le terrain, afin de moduler la sensibilité du système à l'écart mesuré.

Afin de garantir que la commande envoyée au servomoteur reste dans ses limites physiques, l'angle calculé était ensuite borné entre -35° et $+15^\circ$.

Malheureusement, cet algorithme n'a pas été implémenté dans la version finale du projet, car il s'est révélé peu fiable lors des phases de test. Une hypothèse sur cette instabilité est liée au fait que l'algorithme repose uniquement sur la différence entre les distances mesurées à droite et à gauche. Ce fonctionnement suppose implicitement que l'environnement est relativement symétrique ou que les écarts de distance restent modérés. Or, dans certaines portions du circuit, comme les virages serrés, les capteurs peuvent détecter une grande ouverture d'un côté (par exemple bien plus que les 60 cm habituels), ce qui conduit à une correction trop importante de la direction. Ce comportement provoque alors des changements brusques et inadaptés. L'approche s'est donc avérée trop sensible aux variations du tracé et peu adaptée à une navigation fluide dans un environnement réel.

5.4. *Explication du fonctionnement*

Le code dans son entièreté pourra être retrouvé dans l'annexe 1.

1. Détection du feu vert



Avant que la voiture ne démarre, le programme attend la détection du feu vert grâce au capteur de couleur. Cela se fait avec la boucle suivante :

Ici, la voiture reste à l'arrêt tant que la couleur détectée n'est pas le vert. Dès que le capteur couleur détecte le vert, la variable Vert passe à True et le programme sort de la boucle, ce qui déclenche le début du parcours.

2. Lecture continue des distances des capteurs ultrason

Pour surveiller en temps réel les distances à gauche, au centre et à droite, chaque capteur ultrasonique lit sa distance dans un thread séparé :



Cela permet d'avoir des mesures actualisées en permanence, ce qui est essentiel pour la détection rapide des obstacles.

3. Système de log et transmission des données

Le code intègre un système de log qui enregistre et transmet en continu les données importantes du véhicule : vitesses, angles, distances mesurées, détection de la ligne d'arrivée, et mesures électriques. Ceci est réalisé dans un thread dédié :

Un autre thread réinitialise le fichier de log toutes les 30 secondes :

[illegible]

```
def reset_log_file():
    while True:
        time.sleep(30)
        with open("logs_voiture.txt", "w") as log_file:
            log_file.write("")
        print("📄 fichier log réinitialisé.")
```

Ce système permet :

- De garder une trace détaillée de tous les événements du véhicule,
- De transmettre les données à un client connecté,
- De réinitialiser régulièrement les logs pour éviter leur saturation.

4. Détection et esquive des obstacles

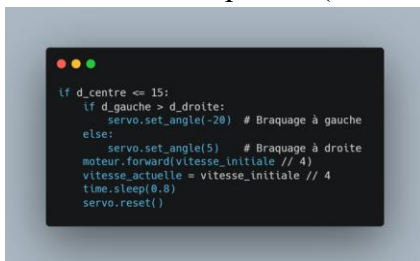
La partie principale de la gestion des obstacles se trouve dans la grande boucle while Vert. À chaque itération, la voiture lit les distances mesurées par les capteurs et décide de la réaction à adopter.

1. Obstacle immédiat (très proche ou anomalie de lecture)

[illegible]

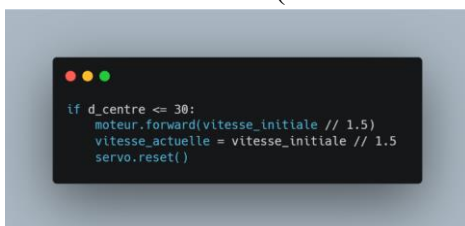
Dans ce cas, la voiture s'arrête immédiatement, recule, puis choisit de tourner du côté où il y a le plus d'espace pour contourner l'obstacle, avance prudemment, puis reprend sa trajectoire normale.

2. Obstacle très proche (moins de 15 cm)



La voiture ralentit fortement et braque rapidement du côté le plus dégagé pour éviter l'obstacle.

3. Obstacle modéré (moins de 30 cm)



Dans ce cas, la voiture réduit simplement sa vitesse sans changer de direction.

4. Obstacle sur les côtés



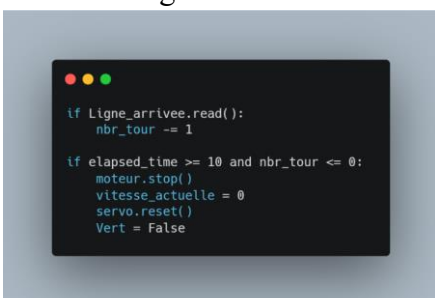
Si un obstacle est détecté à gauche ou à droite, la voiture braque dans la direction opposée pour éviter la collision.

5. Aucun obstacle détecté



Quand il n'y a pas d'obstacle proche, la voiture avance tout droit à sa vitesse initiale.

1. Détection de la ligne d'arrivée



La voiture vérifie en continu si elle détecte la ligne d'arrivée grâce au capteur infrarouge. Si la ligne est détectée après au moins 10 secondes de course, la voiture s'arrête complètement.

6. Problèmes rencontrés

Durant cette semaine de projet, nous avons rencontré plusieurs problèmes que ce soit au niveau du software que du hardware.

- En début de sprint, un manque d'organisation et de planification a entraîné l'absence de points de contact réguliers avec le Product Owner. Par conséquent, aucune question ou clarification n'a pu lui être posée jusqu'à mercredi, ce qui a freiné certaines décisions importantes sur l'orientation du projet.
- L'installation initiale du système Linux n'a pas été faite correctement, en raison d'une mauvaise compréhension des prérequis nécessaires à la configuration du service SSH. Cela a nécessité une réinstallation complète, entraînant une perte de temps considérable et un retard dans les tâches dépendantes de cette configuration.
- À certains moments, la communication entre les membres de l'équipe a été insuffisante. Cela a conduit à des cas de doublons dans le code, certains ayant travaillé sur les mêmes fonctionnalités sans coordination préalable. Ce manque de synchronisation a causé une perte d'efficacité.

- Les réunions Scrum, censées rythmer le projet et garantir un suivi constant, n'ont pas toujours été tenues à temps, ou dans certains cas, pas respectées dans leur forme. Cette situation a nui à la transparence et au suivi de l'avancement réel de chaque membre du groupe.
- Nous avons également rencontré des difficultés technique liées au câblage des modules. Certains branchements étaient incorrects ou mal interprétés, ce qui a généré des erreurs de fonctionnement, voire des dysfonctionnements sur certains capteurs ou composants électroniques.

7. Améliorations et conclusions

Les différents problèmes rencontrés lors de cette semaine de projet pourront être corrigés à l'avenir avec ces solutions :

- Meilleure communication : Avec une meilleure communication nous aurions pu poser nos questions dans les temps au Product Owner afin d'éclaircir certains doutes zones floues dans les différentes user stories. De plus, une meilleure communication nous aurait permis de ne pas coder en double et donc de perdre inutilement du temps et nous aurions pu donc être plus productifs.
- Meilleure compréhension : Nous aurions dû mieux nous renseigner au travers de la documentation afin de moins « foncer la tête baissée » et être plus renseignés sur les difficultés que nous pourrions rencontrer et ainsi mieux y faire face.
- Mieux appliquer la méthodologie Scrum : Une meilleure application de la méthodologie Scrum nous auraient permis d'être plus organisés dans notre travail et donc renforcer notre productivité.

En conclusion, ce projet a été très bénéfique pour tous les membres du groupe. En effet, c'est la première fois que nous appliquons la méthode Scrum dans un véritable projet. Cela nous a permis de mieux planifier, communiquer et organiser notre travail afin de mener à bien ce projet d'envergure. De plus, nous avons appris à développer des tests unitaires afin de s'assurer du bon fonctionnement de notre matériel mais aussi de limiter nos erreurs en utilisant celui-ci et de renforcer la fiabilité de notre code. Nous avons également réutilisé nos connaissances en Python et POO ce qui nous a permis de réaliser un code plus propre, plus modulaire et évolutif.

Malgré les problèmes et difficultés rencontrés, nous avons pu délivrer un produit au travers de cette voiture autonome. Cette voiture est le résultat de la mise en commun de nos connaissances, nos idées et de notre façon de réagir faces aux problèmes.

Pour finir, à la fin de ce projet, nous constatons que nous avons pu améliorer nos connaissances en programmation, mais également notre faculté à appliquer la méthodologie Scrum. Il ne fait aucun doute que ce projet a été très enrichissant pour chacun de nous et nous aidera pour nos futurs projets.

8. Bibliographie

- Syllabus :

NAIZY G., PIETRZAK Y., *La théorie de la gestion de projets*, Haute Ecole en Hainaut à Mons, Année académique 2024-2025.

ROLAND F., *Initiations aux nano-ordinateurs*, Haute Ecole en Hainaut à Mons, Année académique 2024-2025.

DEPRETER J., *Tests unitaires*, Haute Ecole en Hainaut à Mons, Année académique 2024-2025.

- Sources électroniques :

MC HOBBY SPRL, (s. d.), « Capteur distance ultrason – HC-SR04 », *shop.mchobby.be*, [en ligne], <https://shop.mchobby.be/fr/proximite-distance/561-capteur-distance-ultrason-hc-sr04-3232100005617.html>, page consultée le 07 avril 2025.

SIMAC ELECTRONICS GMBH, (2024), « KY-033 Module suiveur de ligne », *sensorkit.joy-it.net*, [en ligne], <https://sensorkit.joy-it.net/fr/sensors/ky-033>, page consultée le 07 avril 2025.

RASPBERRY PI, (s. d.), « Raspberry Pi 3 Model B+ », *rasberrypi.com*, [en ligne], <https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>, page consultée le 07 avril 2025.

CFAURY, (2023), « Capteur de courant et de tension INA 219 », *arduino.blaise-pascal.fr*, [en ligne], <https://arduino.blaise-pascal.fr/capteur-de-courant-et-tension-ina-219/>, page consultée le 07 avril 2025.

GUENEAU C., (s. d.), « Capteur de couleur TCS3472 », *gcworks.fr*, [en ligne], <https://www.gcworks.fr/tutoriel/microbit/CapteurdecouleurTCS3472.html>, page consultée le 07 avril 2025.

JEROME, (2021), « Module L298N : fonctionnement, branchement module », *passionelectronique.fr*, [en ligne], <https://passionelectronique.fr/tutoriel-l298n/>, page consultée le 07 avril 2025.

RASPBERRY PI, (2020), « Comment contrôler un servomoteur avec le Raspberry Pi », *raspberrypi.fr*, [en ligne], <https://raspberrypi.fr/servomoteur-raspberry-pi/#:~:text=Le%20servomoteur%20%3A%20Il%20tourne%20ave,mouvement%20court%20de%20grande%20pr%C3%A9cision>, page consultée le 07 avril 2025.

XUKYO, (2020), « Utilisation d'un module PCA9685 avec Raspberry Pi », *aranacorp.com*, [en ligne], <https://www.aranacorp.com/fr/utilisation-dun-module-pca9685-avec-raspberry-pi/>, page consultée le 07 avril 2025.

9. Annexes

9.1. Programmes complets

Annexe 1 :



<https://github.com/ch4rllystd/GestionDeProjet/blob/main/src/main.py>

9.2. *Gestion du projet*

Annexe 1 Sprint Planning

Sprint planning 1

Dans le cadre du premier sprint, l'équipe a décidé de se concentrer sur les User Stories suivantes : n°1, n°2, n°3 et n°4. Ces tâches ont été sélectionnées en raison de leur importance dans la mise en place des bases du projet, notamment la communication avec le module, la documentation des choix techniques, ainsi que la vérification du bon fonctionnement des capteurs et moteurs.

Sprint planning 2

L'objectif principal de ce sprint était de réaliser les user stories allant de la 2 à la 13.

1. Finalisation de l'implémentation des capteurs : Cette étape est prioritaire, car elle conditionne la suite du développement. Tous les capteurs doivent être intégrés et fonctionnels.
2. Mise en place de la logique de la voiture : Une fois les capteurs opérationnels, nous avons développé la logique de fonctionnement du véhicule, en lien avec les scénarios définis dans les user stories.

Annexe 2 Sprint Rétrospective

Sprint retrospective 1

Durant ce premier sprint, les User Stories numéro 2 et 3 n'ont pas pu être entièrement conclues. Néanmoins, leur progression était déjà bien engagée et leur achèvement paraissait imminent. Cette circonstance nous avait révélé le besoin d'optimiser notre structure organisationnelle. Pour le prochain sprint, nous avons opté pour une approche de travail plus structurée, en séparant les tâches de façon plus détaillée et autonome. Le but était de diminuer les interdépendances au sein de l'équipe pour rendre le travail en commun plus fluide et prévenir les obstacles réciproques.

Sprint retrospective 2

Notre évaluation a révélé une préparation théorique insuffisante concernant le fonctionnement des capteurs, ce qui a directement compromis la qualité de notre démonstration ainsi que la validation des User Stories.

Afin d'optimiser notre efficacité lors des prochains sprints, nous avons identifié quatre axes d'amélioration prioritaires :

Renforcement des compétences techniques : Approfondir notre maîtrise des différentes technologies de capteurs (ultrasons, infrarouge, etc.) pour mieux anticiper leurs contraintes et paramètres d'étalonnage.

Définition rigoureuse des critères d'acceptation : Établir collectivement et précisément les critères de validation pour chaque User Story dès leur conception initiale, éliminant ainsi les ambiguïtés lors de la phase d'évaluation.

Structuration méthodique du travail d'équipe : Implémenter une organisation plus formalisée avec des attributions de rôles explicites et des réunions de synchronisation régulières pour un suivi optimal de l'avancement.

Amélioration de la communication technique : Encourager le partage proactif des connaissances, découvertes et obstacles rencontrés par chaque membre de l'équipe.

Trello :

<https://trello.com/b/A2HLP Eoo/grp7>

